trivial unions (was std::uninitialized<T>)

Document #:	P3074R7 [Latest] [Status]
Date:	2025-02-14
Project:	Programming Language C++
Audience:	EWG

Reply-to:	Barry Revzin <barry.revzin@gmail.com>

Contents

§ 1 Revision History

Since [P3074R6], core wording updates.

Since [P3074R5], core wording updates.

Since [P3074R4], wording changes and adjusted the rule for when a union's destructor is deleted

Since [P3074R3], in St. Louis, EWG had expressed a clear preference for "just make it work":

SF	F	N	A	SA
4	14	3	2	0

over `trivial union`:

SF	F	N	A	SA
0	3	13	4	1

So proposing to make it work and adding implementation experience.

Since [P3074R2], changed to instead propose a language change to unions (with two options) to solve the problems presented

Since [P3074R1], the `std::uninitialized<T>` design was designed in an EWG telecon and the suggestion was made to make this a language feature. Added a section to argue against and re-spelled `std::uninitialized<T>` to be a union instead of a class containing an anonymous union.

Since [P3074R0], originally proposed the function `std::start_lifetime(p)`. R1 adds a new section discussing the uninitialized storage problem, which motivates a change in design to instead propose `std::uninitialized<T>`.

§ 2 Introduction

Consider the following example:

```
template <typename T, size_t N>
struct FixedVector {
    union U { constexpr U() { } constexpr ~U() { } T storage[N]; };
    U u;
    size_t size = 0;

    // note: we are *not* constructing storage
    constexpr FixedVector() = default;

    constexpr ~FixedVector() {
        std::destroy(u.storage, u.storage+size);
    }

    constexpr auto push_back(T const& v) -> void {
        std::construct_at(u.storage + size, v);
        ++size;
    }
};

constexpr auto silly_test() -> size_t {
    FixedVector<std::string, 3> v;
    v.push_back("some sufficiently longer string");
    return v.size;
}
static_assert(silly_test() == 1);
```

This is basically how any static/non-allocating/in-place vector is implemented: we have some storage, that we *definitely do not value initialize* and then we steadily construct elements into it.

The problem is that the above does not work (although there is implementation divergence - MSVC and EDG accept it and GCC did accept it even up to 13.2, but GCC trunk and Clang reject).

Getting this example to work would allow `std::inplace_vector` ([P0843R14]) to simply work during `constexpr` time for all times (instead of just trivial ones), and was a problem briefly touched on in [P2747R0].

§ 2.1 The uninitialized storage problem

A closely related problem to the above is: how do you do uninitialized storage? The straightforward implementation would be to do:

```
template <class T>
struct BufferStorage {
private:
    alignas(T) unsigned char buffer[sizeof(T)];

public:
    // accessors
};
```

This approach generally works, but it has two limitations:

1. it cannot work in `constexpr` and that's likely a fundamental limitation that will never change, and
2. it does not quite handle overlapping objects correctly.

What I mean by the second one is basically given this structure:

```
struct Empty { };

struct Sub : Empty {
    BufferStorage<Empty> buffer_storage;
};
```

If we initialize the `Empty` that `buffer_storage` is intended to have, then `Sub` has two subobjects of type `Empty`. But the compiler doesn't really… know that, and doesn't adjust them accordingly. As a result, the `Empty` base class subobject and the `Empty` initialized in `buffer_storage` are at the same address, which violates the rule that all objects of one type are at unique addresses.

An alternative approach to storage is to use a `union`:

```
template <class T>
struct UnionStorage {
```

```
private:
  union { T value; };

public:
  // accessors
};

struct Sub : Empty {
    UnionStorage<Empty> union_storage;
};
```

Here, now the compiler knows for sure there is an `Empty` in `union_storage` and will lay out the types appropriately. See also gcc bug 112591.

So it seems that the `UnionStorage` approach is strictly superior: it will work in constexpr and it lays out overlapping types properly. But it has limitations of its own. As with the `FixedVector` example earlier, you cannot just start the lifetime of `value`. But also in this case we run into the `union` rules for special member functions: a special member of a `union`, by default, is either trivial (if that special member for all alternatives is trivial) or deleted (otherwise). Which means that `UnionStorage<std::string>` has both its constructor and destructor *deleted*.

We can work around this by simply adding an empty constructor and destructor (as shown earlier as well):

```
template <class T>
struct UnionStorage2 {
private:
  union U { U() { } ~U() { } T value; };
  U u;

public:
  // accessors
};
```

This is a fundamentally weird concept since `U` there has a destructor that does nothing (and given that this is a class to be used for uninitialized storage), it *should* do nothing - that's correct. But that destructor still isn't trivial. And it turns out there is still a difference between "destructor that does nothing" and "trivial destructor":

Trivially Destructible	Non-trivially Destructible

`struct A { };`
 	
`auto alloc_a(int n) -> A* { return new A[n]; }`
`auto del(A* p) -> void { delete [] p; }`
	`struct B { ~B() { } };`
	
	`auto alloc_b(int n) -> B* { return new B[n]; }`
	`auto del(B* p) -> void { delete [] p; }`

Trivially Destructible	Non-trivially Destructible

`alloc_a(int):`
`  movsx  rdi, edi`
`  jmp  operator new[]`
`  (unsigned long)`
 	
`del(A*):`
`  test  rdi, rdi`
`  je  .L3`
`  jmp  operator delete[]`
`  (void*)`
`.L3:`
`  ret`
	`alloc_b(int):`
	`  movabs  rax, 9223372036854775800`
	`  push  rbx`
	`  movsx  rbx, edi`
	`  cmp  rax, rbx`
	`  lea  rdi, [rbx+8]`
	`  mov  rax, -1`
	`  cmovb  rdi, rax`
	`  call  operator new[](unsigned`
	`  long)`
	`  mov  QWORD PTR [rax], rbx`
	`  add  rax, 8`
	`  pop  rbx`
	`  ret`
	
	`del(B*):`
	`  test  rdi, rdi`
	`  je  .L9`
	`  mov  rax, QWORD PTR [rdi-8]`
	`  sub  rdi, 8`
	`  lea  rsi, [rax+8]`
	`  jmp  operator delete[](void*,`
	`  unsigned long)`
	`.L9:`
	`  ret`

That's a big difference in code-gen, due to the need to put a cookie in the allocation so that the corresponding `delete[]` knows how many elements there so that their destructors (even though they do nothing!) can be invoked.

While the union storage solution solves some language problems for us, the buffer storage solution can lead to more efficient code - because `StorageBuffer<T>` is always trivially destructible. It would be nice if he had a good solution to all of these problems - and that solution was also the most efficient one.

§ 3 Design Space

There are several potential solutions in this space:

1. a library solution (add a `std::uninitialized<T>`)
2. a language solution (add some annotation to members to mark them uninitialized, as distinct from `union`s)
3. just make it work (change the union rules to implicitly start the lifetime of the first alternative, if it's an implicit-lifetime type)
4. introduce a new kind of union
5. provide an explicit function to start lifetime of a union alternative (`std::start_lifetime`).

The first revision of this paper ([P3074R0]) proposed that last option. However, with the addition of the overlapping subobjects problem and the realization that the union solution has overhead compared to the buffer storage solution, it would be more desirable to solve both problems in one go. That is, it's not enough to just start the lifetime of the alternative, we also want a trivially constructible/destructible solution for uninitialized storage.

[P3074R1] and [P3074R2] proposed the first solution (`std::uninitialized<T>`). [P3074R3] proposed either the third or fourth. This revision (R4) proposes specifically the third (just make it work).

Let's go over some of the solutions.

3.1 A library type: `std::uninitialized<T>`

We could introduce another magic library type, `std::uninitialized<T>`, with an interface like:

```
template <typename T>
struct uninitialized {
    union { T value; };
};
```

As basically a better version of `std::aligned_storage`. Here is storage for a `T`, that implicitly begins its lifetime if `T` is an implicit-lifetime-type, but otherwise will not actually initialize it for you - you have to do that yourself. Likewise it will not destroy it for you, you have to do that yourself too. This type would be specified to always be trivially default constructible and trivially destructible. It would be trivially copyable if `T` is trivially copyable, otherwise not copyable.

`std::inplace_vector<T, N>` would then have a `std::uninitialized<T[N]>` and go ahead and `std::construct_at` (or, with [P2747R2], simply placement-new) into the appropriate elements of that array and everything would just work.

Because the language would recognize this type, this would also solve the overlapping objects problem.

3.2 A language annotation

During the EWG telecon in January 2023, the suggestion was made that instead of a magic library type like `std::uninitialized<T>`, we could instead have some kind of language annotation to achieve the same effect.

For example:

```
template <typename T, size_t N>
struct FixedVector {
    // as a library feature
    std::uninitialized<T[N]> lib;

    //as a language feature, something like this
    for storage T lang[N];
```

```
    T storage[N] = for lang;
    T storage[N] = void;
    uninitialized T lang[N];

    size_t size = 0;
};
```

The advantage of the language syntax is that you can directly use `lang` - you would placement new onto `lang[0]`, you read from `lang[1]`, etc, whereas with the library syntax you have to placement new onto `lib.value[0]` and read from `lib.value[1]`, etc.

In that telecon, there was preference (including by me) for the language solution:

SF	F	N	A	SA
5	4	4	2	1

However, an uninitialized object of type `T` really isn't the same thing as a `T`. `decltype(lang)` would have to be `T`, any kind of (imminent) reflection over this type would give you a `T`. But there might not actually be a `T` there yet, it behaves like a `union { T; }` rather than a `T`, so spelling it `T` strikes me as misleading.

We would have to ensure that all the other member-wise algorithms we have today (the special member functions and the comparisons) use the "uninitialized `T`" meaning rather than the `T` meaning. And with reflection, that also means all future member-wise algorithms would have to account for this also - rather than rejecting `union`s. This seems to open the door to a lot of mistakes.

The syntactic benefits of the language syntax are nice, but this is a rarely used type for specific situations - so having slightly longer syntax (and really, `lib.value` is not especially cumbersome) is not only not a big downside here but could even be viewed as a benefit.

For this reason, R2 of this paper still proposed `std::uninitialized<T>` as the solution in preference to any language annotation. This did not go over well in Tokyo, where again there was preference for the language solution:

SF	F	N	A	SA
6	7	3	4	2

This leads to…

§ 3.3 Just make it work

Now, for the `inplace_vector` problem, today's `union` is insufficient:

```
template <typename T, size_t N>
struct FixedVector {
    union { T storage[N]; };
```

```
    size_t size = 0;
};
```

Similarly a simple `union { T storage; }` is insufficient for the uninitialized storage problem.

There are three reasons for this:

1. the default constructor can be deleted (this can be easily worked around though)
2. the default constructor does not start the lifetime of implicit lifetime types
3. the destructor can be deleted (this can be worked around by providing a no-op destructor, which has ABI cost that cannot be worked around)

However, what if instead of coming up with a solution for these problems, we just… made it work?

That is, change the union rules as follows:

member	status quo	new rule
default constructor (absent default member initializers)	If all the alternatives are trivially default constructible, trivial. Otherwise, deleted.	Unconditionally trivial If the first alternative has implicit-lifetime type, starts the lifetime of that alternative and sets it as the active member (no initialization is performed).
destructor	If all the alternatives are trivially destructible, trivial. Otherwise, deleted.	Unconditionally trivial.

This isn't quite a *minimal* extension, we could make it even more minimal by only allowing a trivial default constructor and trivial destructor for implicit-lifetime types, as in:

```
// default constructor and destructor are both deleted
union U1 { std::string s; };

// default constructor and destructor are both trivial
union U2 { std::string a[1]; };
```

But that doesn't seem like a useful distinction to make. It's also actively harmful — for uninitialized storage, we really want trivial construction/destruction. And it would be nice to not have to resort to having members of type `T[1]` instead of `T` to achieve this.

Simply stating that the default constructor (absent default member initializers) and destructor are always trivial is a simple rule.

§ 3.4 Trivial Unions

What if we introduced a new kind of union, with special annotation? That is:

```
template <typename T, size_t N>
struct FixedVector {
    trivial union { T storage[N]; };
    size_t size = 0;
};
```

With the rule that a trivial union is just always trivially default constructible, trivially destructible, and, if the first alternative is implicit-lifetime, starts the lifetime of that alternative (and sets it to be the active member).

This is a language solution that doesn't have any of the consequences for memberwise algorithms - since we're still a `union`. It provides a clean solution to the uninitialized storage problem, the aliasing problem, and the constexpr `inplace_vector` storage problem. Without having to deal with potentially changing behavior of existing unions.

This brings up the question about default member initializers. Should a `trivial union` be allowed to have a default member initializer? I don't think so. If you're initializing the thing, it's not really uninitialized storage anymore. Use a regular union.

An alternative spelling for this might be `uninitialized union` instead of `trivial union`. An alternative alternative would be to instead provide a different way of declaring the constructor and destructor:

```
union U {
  U() = trivial;
  ~U() = trivial;

  T storage[N];
};
```

This is explicit (unlike just making it work), but seems unnecessary much to type compared to a single `trivial` token - and these things really aren't orthogonal. Plus it wouldn't allow for anonymous trivial unions, which seems like a nice usability gain.

§ 3.5 Existing Practice

There are three similar features in other languages that I'm aware of.

Rust has `MaybeUninit<T>` which is similar to what's described here as `std::uninitialized<T>`.

Kotlin has a `lateinit var` language feature, which is similar to some kind of language annotation (although additionally allows for checking whether it has been initialized, which the language feature would not provide).

D has the ability to initialize a variable to `void`, as in `int x = void;` This leaves `x` uninitialized. However, this feature only affects construction - not destruction. A member `T[N] storage = void;` would leave the array uninitialized, but would destroy the whole array in the destructor. So not really suitable for this particular purpose.

§ 3.6 St. Louis Meeting, 2024

In St. Louis, we discussed a previous revision of this paper ([P3074R3]), specifically the trivial union and just make it work designs. There, EWG expressed a clear preference for "just make it work":

SF	F	N	A	SA
4	14	3	2	0

over `trivial union`:

SF	F	N	A	SA
0	3	13	4	1

So this paper proposes the more favorable design.

§ 3.7 Just to Double Check

During Core review in Wrocław, there was a desire to make it clear that this proposal can change code that was previously ill-formed to instead become undefined behavior:

```
union U {
    std::string s;
};

auto f() -> std::string {
    U u;
    return u.s;
}
```

Status quo is that this program is ill-formed because `U`'s constructor and destructor are defined as deleted. With this proposal, they become trivial — constructing `u` is valid but `u.s` is uninitialized, which is then returned.

In practice, I don't think this is a huge issue since users can simply "fix" the issue of the deleted constructor and destructor by adding `U() { }` and `~U() { }` — and now we're back to the exact same problem anyway.

§ 3.8 Constructor/Destructor Intention Matching

Consider:

```
union U1 {
    std::string s = "this";
};

union U2 {
    U2() : s("or that") { }
    std::string s;
};

union U3 {
  string s;
  U3* next = nullptr;
};

union U4 {
  string s;
  int i;
};
```

Now, `U4` is the simple case. Our constructor is doing no initialization, so it's reasonable that the corresponding destructor also does nothing. But for the other three, constructing one of these unions actually does something. So what should the destructor look like?

Core in Wrocław suggested that the constructor and destructor should really match. That is, if a `union` has a variant with a non-trivial destructor and that union has non-trivial default constructor (either by having a user-provided default constructor or by having a default member initializer), then we should retain the original rule and keep the deleted destructor.

A good principle to follow, I think, is that this code:

```
{
    // for some union U
    U u;
}
```

Should either not compile (because the destructor is deleted — or more boringly because `U` isn't default constructible) or be fine (because either we know the initialization is okay and thus the destructor can be trivial, or we forced the user to take care of it).

Thus the rule: the defaulted destructor for a union is defined as deleted if either there is a user-provided default constructor or there is a variant member with a default member initializer and that variant member has a destructor that is either inaccessible or deleted.

§ 4 Proposal

This paper proposes to just make it work. That is:

— The default constructor, absent default member initializers, is always trivial. If the first alternative is an implicit-lifetime time, it begins its lifetime and becomes the active alternative.

— The defaulted destructor is deleted if either (a) the union has a user-provided default constructor or (b) there exists a variant alternative that has a default member initializer and that member's destructor is either deleted or inaccessible. Otherwise, the destructor is trivial.

All other special members remain unchanged. The behavior for a few examples looks like this:

```
// trivial default constructor (does not start lifetime of s)
// trivial destructor
// (status quo: deleted default constructor and destructor)
union U1 { string s; };

// non-trivial default constructor
// deleted destructor
// (status quo: deleted destructor)
union U2 { string s = "hello"; }

// trivial default constructor
// starts lifetime of s
// trivial destructor
// (status quo: deleted default constructor and destructor)
union U3 { string s[10]; }

// non-trivial default constructor (initializes next)
// trivial destructor
// (status quo: deleted destructor)
union U4 { string s; U4* next = nullptr; };
```

Note that just making work will change some code from ill-formed to well-formed, but seems unlikely to change the meaning of any existing already-valid code.

§ 4.1 Implementation Experience

I implemented this paper in clang, it was not difficult. The clang tests that exist to check for the existing `union` behavior (i.e. that a union with an alternative with a non-trivial or no default constructor has a deleted default constructor) now fail, as expected. But something like this now passes (clang already implements `constexpr` placement new — the status quo is that referencing `&s[0]` is ill-formed because `s` has not began its lifetime):

```
constexpr int f1() {
    union { int s[4]; };
    new (&s[0]) int(1);
    new (&s[1]) int(2);
    new (&s[2]) int(3);
    return s[0] + s[1] + s[2];
}
static_assert(f1() == 6);
```

I was able to compile Clang with this update successfully, which wasn't particularly surprising since this change is entirely about making existing ill-formed code valid — and the Clang implementation is already valid code.

§ 4.2 Language Wording

Change 11.4.5.2 [class.default.ctor]/2-3. [Editor's note: The third and fourth bullets can be removed because such cases become trivially default constructible too]

2 A defaulted default constructor for class `X` is defined as deleted if

(2.1) — any non-static data member with no default member initializer ([class.mem]) is of reference type,

(2.2) — `X` is a non-union class and any non-variant non-static data member of const-qualified type (or possibly multi-dimensional array thereof) with no brace-or-equal-initializer is not const-default-constructible ([dcl.init]),

(2.3) — ~~`X` is a union and all of its variant members are of const-qualified type (or possibly multi-dimensional array thereof),~~

(2.4) — ~~`X` is a non-union class and all members of any anonymous union member are of const-qualified type (or possibly multi-dimensional array thereof)~~,

(2.5) — any non-variant potentially constructed subobject, except for a non-static data member with a brace-or-equal-initializer ~~or a variant member of a union where another non-static data member has a brace-or-equal-initializer~~, has class type `M` (or possibly multi-dimensional array thereof) and overload resolution ([over.match]) as applied to find `M`'s corresponding constructor ~~either~~ does not result in a usable candidate ([over.match.general]) ~~or, in the case of a variant member, selects a non-trivial function,~~ or

(2.6) — any potentially constructed subobject *`S`* has class type `M` (or possibly multidimensional array thereof), ~~and~~ `M` has a destructor that is deleted or inaccessible from the defaulted default constructor, and either *`S`* is non-variant or *`S`* has a default member initializer.

3 A default constructor for a class `X` is *trivial* if it is not user-provided and if~~:~~

(3.1) — ~~its class~~ `X` has no virtual functions ([class.virtual]) and no virtual base classes ([class.mi]), and

(3.2) — no non-static data member of ~~its class~~ `X` has a default member initializer ([class.mem]), and

(3.3) — all the direct base classes of ~~its class~~ `X` have trivial default constructors, and

(3.4) — either `X` is a union or for all the non-variant non-static data members of ~~its class~~ `X` that are of class type (or array thereof), each such class has a trivial default constructor.

Otherwise, the default constructor is *non-trivial*.

4 If a default constructor of a union-like class `X` is trivial, then for each union `U` that is either `X` or an anonymous union member of `X`, if the first variant member, if any, of `U` has implicit-lifetime type ([basic.types.general]), the default constructor of `X` begins the lifetime of that member if it is not the active member of its union. [*Note 1:* It is already the active member if `U` was value-initialized. — *end note*] ~~An~~ Otherwise, an implicitly-defined ([dcl.fct.def.default]) default constructor performs the set of initializations of the class that would be performed by a user-written default constructor for that class with no ctor-initializer ([class.base.init]) and an empty compound-statement.

Change 11.4.7 [class.dtor]/7-8:

7 A defaulted destructor for a class `X` is defined as deleted if~~:~~

(7.1) — `X` is a non-union class and any non-variant potentially constructed subobject has class type `M` (or possibly multi-dimensional array thereof) ~~and~~ where `M` has a destructor that is deleted or is inaccessible from the defaulted destructor ~~or, in the case of a variant member, is non-trivial~~,

(7.x) — `X` is a union and

(7.x.1) — overload resolution to select a constructor to default-initialize an object of type `X` either fails or selects a constructor that is either deleted or not trivial, or

(7.x.2) — `X` has a variant member `V` of class type `M` (or possibly multi-dimensional array thereof) where `V` has a default member initializer and `M` has a destructor that is non-trivial,

(7.2) — or, for a virtual destructor, lookup of the non-array deallocation function results in an ambiguity or in a function that is deleted or inaccessible from the defaulted destructor.

8 A destructor for a class `X` is *trivial* if it is not user-provided and if~~:~~

(8.1) — the destructor is not virtual,

(8.2) — all of the direct base classes of ~~its class~~ `X` have trivial destructors, and

(8.3) — either `X` is a union or for all of the non-variant non-static data members of ~~its class~~ `X` that are of class type (or array thereof), each such class has a trivial destructor.

Update the note in 11.5.1 [class.union.general]/4:

4 A union can have member functions (including constructors and destructors), but it shall not have virtual ([class.virtual]) functions. A union shall not have base classes. A union shall not be used as a base class. If a union contains a non-static data member of reference type, the program is ill-formed.

[*Note 3:* ~~Absent default member initializers ([class.mem]), if~~ If any non-static data member of a union has a non-trivial ~~default constructor ([class.default.ctor]),~~ copy constructor, move constructor ([class.copy.ctor]), copy assignment operator, or move assignment operator ([class.copy.assign]), ~~or destructor ([class.dtor]),~~ the corresponding member function of the union must be user-provided or it will be implicitly deleted ([dcl.fct.def.delete]) for the union.

[*Example 1:* Consider the following union:

```
union U {
  int i;
  float f;
  std::string s;
};
```

Since `std::string` ([string.classes]) declares non-trivial versions of all of the special member functions, `U` will have an implicitly deleted ~~default constructor,~~ copy/move constructor~~,~~ and copy/move assignment operator~~, and destructor~~. ~~To use `U`, some or all of these member functions must be user-provided.~~ The default constructor and destructor of `U` are both trivial even though `std::string` has a non-trivial default constructor and a non-trivial destructor — *end example*]

— *end note*]

§ 4.3 Library Wording

While this paper was in flight, [P0843R14] was moved, which had to work around the lack of ability to actually support non-trivial types during constant evaluation. Since this paper now provides that, might as well fix the library to account for the new functionality.

Strike 23.3.14.1 [inplace.vector.overview]/4:

3 For any `N`, `inplace_vector<T, N>::iterator` and `inplace_vector<T, N>::const_iterator` meet the constexpr iterator requirements.

4 ~~For any `N>0`, if `is_trivial_v<T>` is `false`, then no `inplace_vector<T, N>` member functions are usable in constant expressions.~~

§ 4.4 Feature-Test Macro

Add a new macro to 15.11 [cpp.predefined]:

```
__cpp_trivial_union 2025XXL
```

Add a new feature-test macro for `constexpr <inplace_vector>` in 17.3.2 [version.syn]:

```
+  #define  __cpp_lib_constexpr_inplace_vector  2025XXL  //  also  in
       <inplace_vector>
```

§ 5 References

[P0843R14] Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel. 2024-06-26. inplace_vector.
https://wg21.link/p0843r14

[P2747R0] Barry Revzin. 2022-12-17. Limited support for constexpr void*.
https://wg21.link/p2747r0

[P2747R2] Barry Revzin. 2024-03-19. constexpr placement new.
https://wg21.link/p2747r2

[P3074R0] Barry Revzin. 2023-12-15. constexpr union lifetime.
https://wg21.link/p3074r0

[P3074R1] Barry Revzin. 2024-01-30. std::uninitialized<T>.
https://wg21.link/p3074r1

[P3074R2] Barry Revzin. 2024-02-13. std::uninitialized<T>.
https://wg21.link/p3074r2

[P3074R3] Barry Revzin. 2024-04-14. trivial union (was std::uninitialized<T>).
https://wg21.link/p3074r3

[P3074R4] Barry Revzin. 2024-09-10. trivial unions (was std::uninitialized<T>).
https://wg21.link/p3074r4

[P3074R5] Barry Revzin. 2024-12-17. trivial unions (was std::uninitialized<T>).
https://wg21.link/p3074r5

[P3074R6] Barry Revzin. 2025-02-13. trivial `unions` (was `std::uninitialized<T>`).
https://wg21.link/p3074r6